

ASSIGNMENT – 14.2

AI ASSISTED CODING

NAME : L.NAVYA

HALLTICKECT NO : 2403A51308

BATCH NUMBER : 01

COURSE CODE : 24CS002PC215

PROGRAM NAME : B.TECH

YEAR/SEM : 2ND AND 3RD

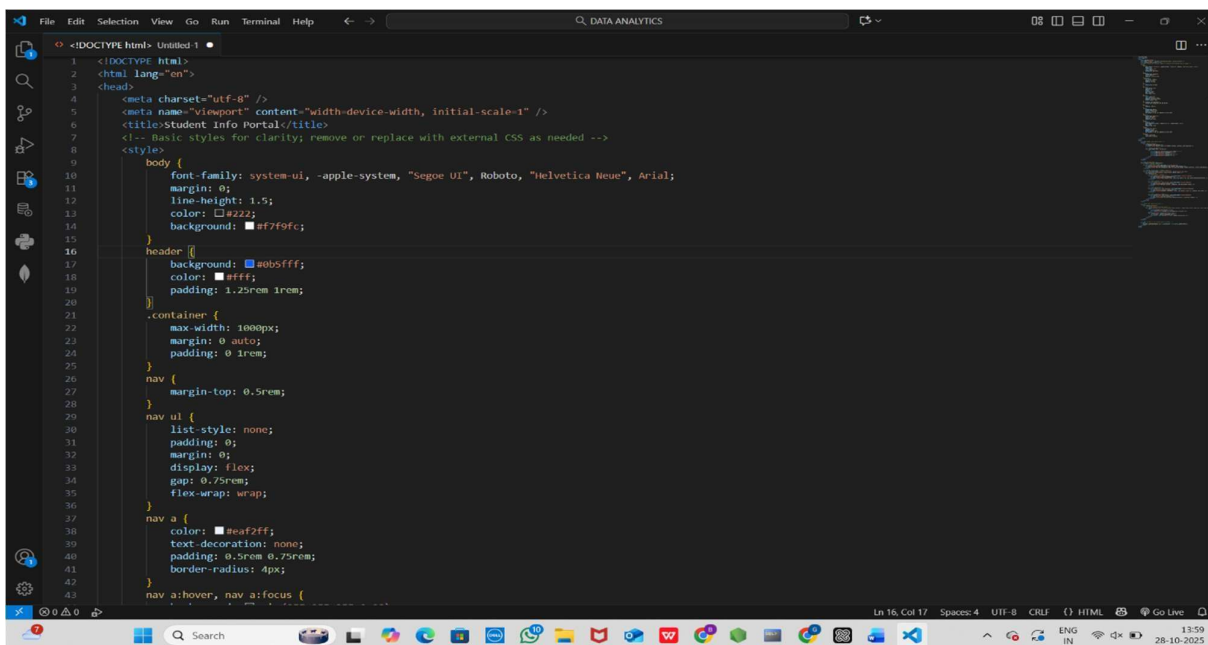
TASK 1 :

Ask AI to generate a simple HTML homepage for a "Student Info Portal" with a header, navigation menu, and footer.

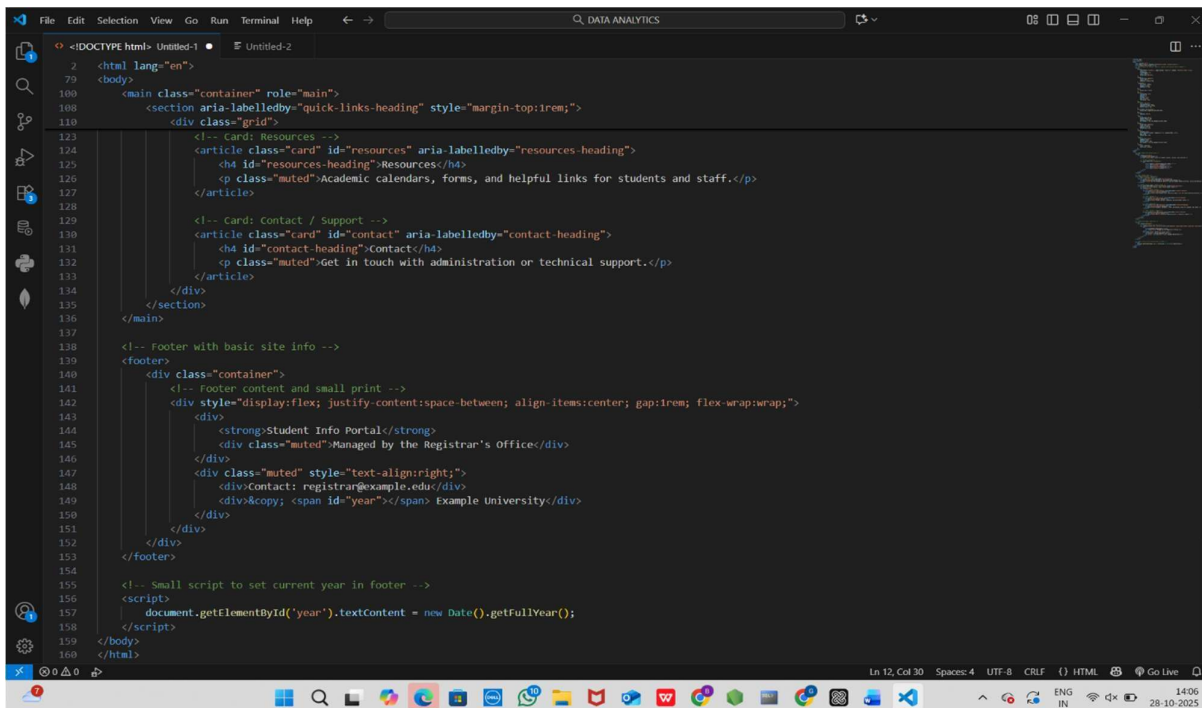
PROMPT:

Generate a simple HTML homepage for a "Student Info Portal" that includes a <header>, <nav>, and <footer>. Ensure the code has clean indentation, proper tags, and includes comments for clarity.

CODE :



```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="utf-8" />
5   <meta name="viewport" content="width=device-width, initial-scale=1" />
6   <title>Student Info Portal</title>
7   <!-- Basic styles for clarity; remove or replace with external CSS as needed -->
8   <style>
9
10    body {
11      font-family: system-ui, -apple-system, "Segoe UI", Roboto, "Helvetica Neue", Arial;
12      margin: 0;
13      line-height: 1.5;
14      color: #a22222;
15      background-color: #f7f9fc;
16    }
17    header {
18      background-color: #a0b5ff;
19      color: #ffff;
20      padding: 1.25rem 1rem;
21    }
22    .container {
23      max-width: 1000px;
24      margin: 0 auto;
25      padding: 0 1rem;
26    }
27    nav {
28      margin-top: 0.5rem;
29    }
30    nav ul {
31      list-style: none;
32      padding: 0;
33      margin: 0;
34      display: flex;
35      gap: 0.75rem;
36      flex-wrap: wrap;
37    }
38    nav a {
39      color: #00a2f2;
40      text-decoration: none;
41      padding: 0.5rem 0.75rem;
42      border-radius: 4px;
43    }
44    nav a:hover, nav a:focus {
45
```

A screenshot of a code editor window with a dark theme. The editor shows HTML code for a student portal homepage. The code includes a header section with a navigation bar, a main content area with two cards (Resources and Contact/Support), and a footer section with site information and a small script to set the current year. The code is well-structured with proper indentation and comments. The editor's interface includes a menu bar at the top, a toolbar on the left, and a status bar at the bottom.

```
<!DOCTYPE html>
<html lang="en">
<body>
  <main class="container" role="main">
    <section aria-labelledby="quick-links-heading" style="margin-top:1rem;">
      <div class="grid">
        <!-- Card: Resources -->
        <article class="card" id="resources" aria-labelledby="resources-heading">
          <h4 id="resources-heading">Resources</h4>
          <p class="muted">Academic calendars, forms, and helpful links for students and staff.</p>
        </article>
        <!-- Card: Contact / Support -->
        <article class="card" id="contact" aria-labelledby="contact-heading">
          <h4 id="contact-heading">Contact</h4>
          <p class="muted">Get in touch with administration or technical support.</p>
        </article>
      </div>
    </section>
  </main>
  <!-- Footer with basic site info -->
  <footer>
    <div class="container">
      <!-- Footer content and small print -->
      <div style="display:flex; justify-content:space-between; align-items:center; gap:1rem; flex-wrap:wrap;">
        <div>
          <strong>Student Info Portal</strong>
          <div class="muted">Managed by the Registrar's Office</div>
        </div>
        <div class="muted" style="text-align:right;">
          <div>Contact: registrar@example.edu</div>
          <div>&copy; <span id="year"></span> Example University</div>
        </div>
      </div>
    </div>
  </footer>
  <!-- Small script to set current year in footer -->
  <script>
    document.getElementById('year').textContent = new Date().getFullYear();
  </script>
</body>
</html>
```

OUTPUT:

- HTML code with <header>, <nav>, <footer>.
- Clean indenta on, proper tags, and comments.

TASK 2:

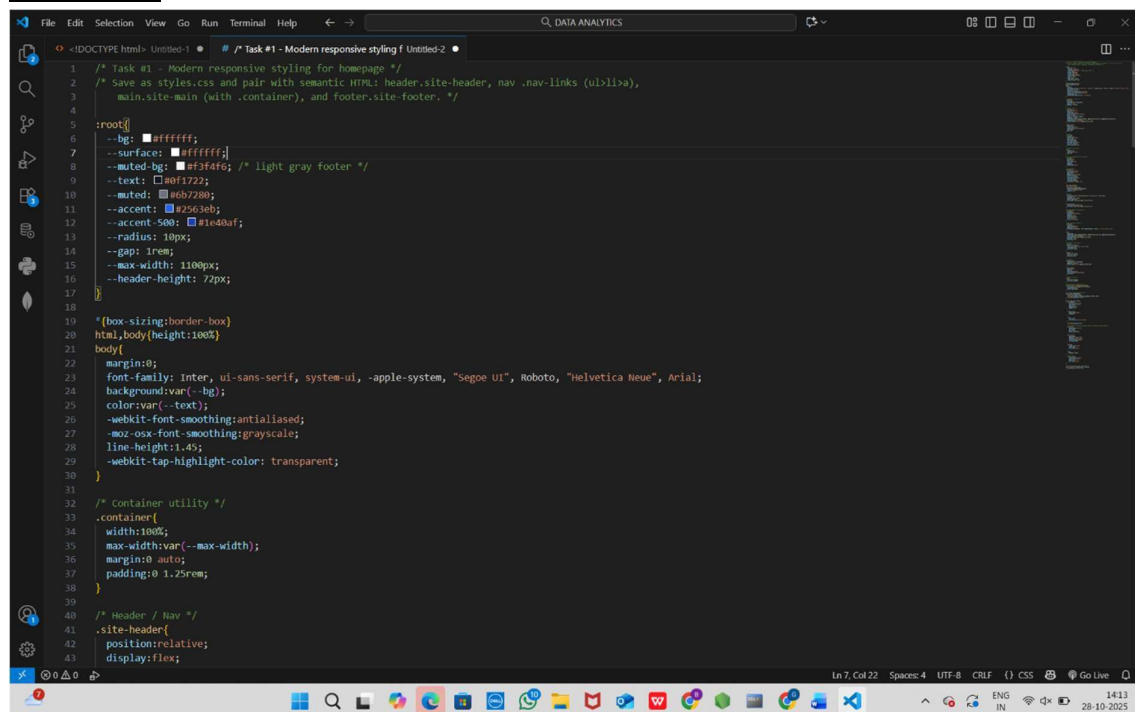
Use AI to add CSS styling to Task #1 homepage for:

- Responsive naviga on bar.
- Centered content sec on.
- Footer with light gray background

PROMPT:

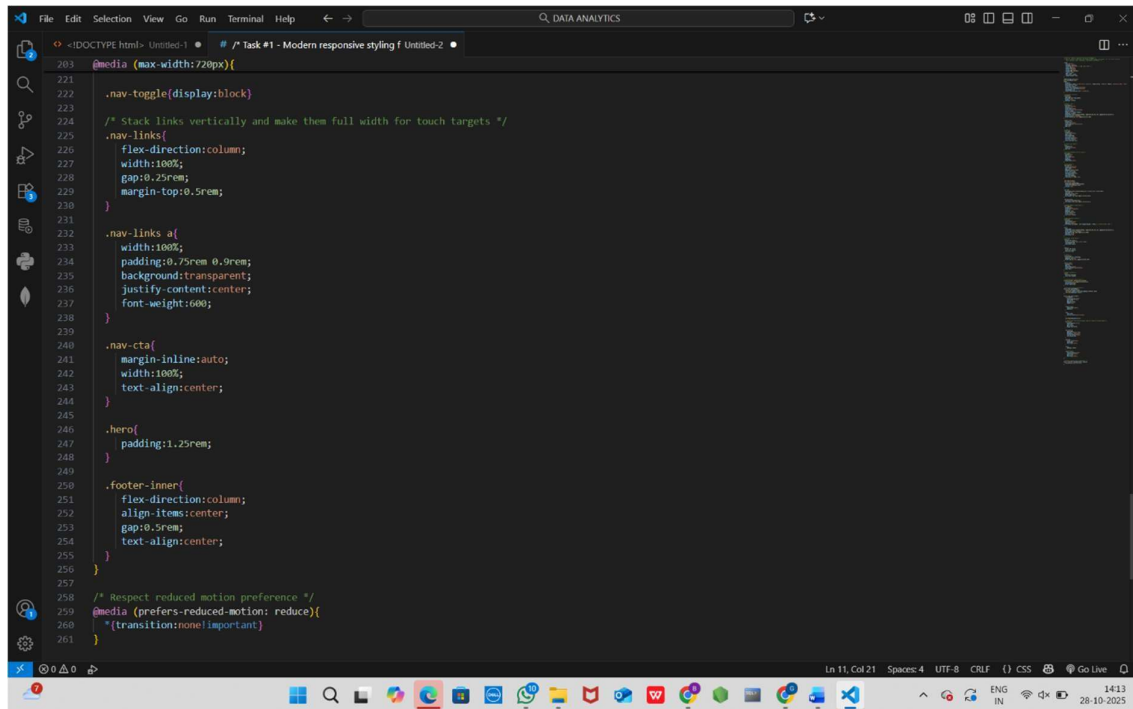
“Add modern CSS styling to Task #1 homepage with a responsive navigation bar (flex or grid), a centered main content section, and a footer with a light gray background. Make sure it’s mobilefriendly and visually clean.”

CODE:

A screenshot of a Visual Studio Code editor window. The title bar shows 'DATA ANALYTICS'. The editor has two tabs: '<DOCTYPE html> Untitled-1' and 'Task #1 - Modern responsive styling f Untitled-2'. The active tab shows CSS code. The code includes comments for styling a homepage, a container utility, and a header/nav section. The CSS uses a root variable for colors and a container utility for layout. The footer is styled with a light gray background. The code is as follows:

```
1 /* Task #1 - Modern responsive styling for homepage */
2 /* Save as styles.css and pair with semantic HTML: header.site-header, nav.nav-links (ul>li>a),
3    main.site-main (with .container), and footer.site-footer. */
4
5 :root{
6   --bg: #ffffff;
7   --surface: #ffffff;
8   --muted-bg: #f3f4f6; /* light gray footer */
9   --text: #000000;
10  --muted: #6b7280;
11  --accent: #2563eb;
12  --accent-500: #1ed761;
13  --radius: 10px;
14  --gap: 1rem;
15  --max-width: 1100px;
16  --header-height: 72px;
17
18
19  /* (box-sizing: border-box)
20  html, body { height: 100%;
21
22  body {
23    margin: 0;
24    font-family: Inter, ui-sans-serif, system-ui, -apple-system, "Segoe UI", Roboto, "Helvetica Neue", Arial;
25    background: var(--bg);
26    color: var(--text);
27    -webkit-font-smoothing: antialiased;
28    -moz-osx-font-smoothing: grayscale;
29    line-height: 1.45;
30    -webkit-tap-highlight-color: transparent;
31  }
32
33  /* Container utility */
34  .container {
35    width: 100%;
36    max-width: var(--max-width);
37    margin: 0 auto;
38    padding: 0 1.25rem;
39  }
40
41  /* Header / Nav */
42  .site-header {
43    position: relative;
44    display: flex;
```

The status bar at the bottom shows 'Ln 7, Col 22', 'Spaces: 4', 'UTF-8', 'CRLF', '() CSS', and 'Go Live'. The system tray at the bottom right shows the date '28-10-2025' and time '14:13'.



The screenshot shows a VS Code editor window with two tabs: 'Untitled-1' and 'Task #1 - Modern responsive styling f Untitled-2'. The active tab contains CSS code for a responsive design. The code includes a media query for screens with a maximum width of 720px, which sets the display of a toggle to 'block' and stacks links vertically. It also defines styles for navigation links, a call-to-action button, a hero section, and a footer. A second media query is provided for users who have reduced motion preference, setting transitions to 'none'.

```
203 @media (max-width:720px){
221
222   .nav-toggle{display:block}
223
224   /* Stack links vertically and make them full width for touch targets */
225   .nav-links{
226     flex-direction:column;
227     width:100%;
228     gap:0.25rem;
229     margin-top:0.5rem;
230   }
231
232   .nav-links a{
233     width:100%;
234     padding:0.25rem 0.9rem;
235     background:transparent;
236     justify-content:center;
237     font-weight:600;
238   }
239
240   .nav-cta{
241     margin-inline:auto;
242     width:100%;
243     text-align:center;
244   }
245
246   .hero{
247     padding:1.25rem;
248   }
249
250   .footer-inner{
251     flex-direction:column;
252     align-items:center;
253     gap:0.5rem;
254     text-align:center;
255   }
256
257
258   /* Respect reduced motion preference */
259   @media (prefers-reduced-motion: reduce){
260     *(transition:none!important)
261   }
262 }
```

OUTPUT:

- HTML + CSS combined.
- AI explains how CSS classes apply.
- AI refactors with with open() and try-except.

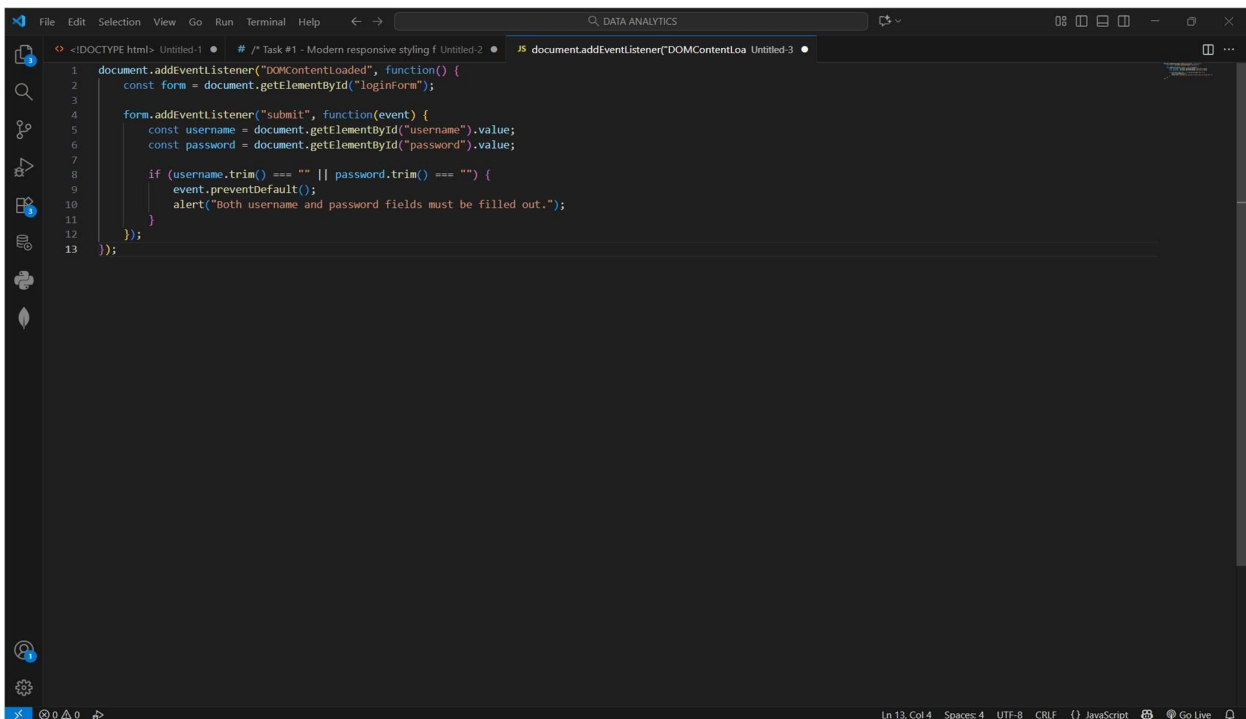
TASK 3:

Prompt AI to generate a JS script that validates a simple login form (non-empty username/password).

PROMPT:

“Generate a JavaScript script that validates a simple login form, ensuring the username and password fields are not empty before submission.”

CODE:

A screenshot of a code editor window with a dark theme. The editor shows a JavaScript file named 'document.addEventListener("DOMContentLoaded", function() {'. The code implements a form validation function. It first gets the form element by ID 'loginForm'. Then, it adds an event listener for the 'submit' event. Inside this listener, it gets the values of the 'username' and 'password' input fields. It then checks if either field is empty (after trimming). If both are empty, it calls 'event.preventDefault()' and shows an alert message: 'Both username and password fields must be filled out.'. The code is as follows:

```
1 document.addEventListener("DOMContentLoaded", function() {
2   const form = document.getElementById("loginForm");
3
4   form.addEventListener("submit", function(event) {
5     const username = document.getElementById("username").value;
6     const password = document.getElementById("password").value;
7
8     if (username.trim() === "" || password.trim() === "") {
9       event.preventDefault();
10      alert("Both username and password fields must be filled out.");
11    }
12  });
13 });
```

The editor interface includes a sidebar on the left with icons for Explorer, Search, Source Control, and Run and Debug. The bottom status bar shows 'Ln 13, Col 4', 'Spaces: 4', 'UTF-8', 'CRLF', and 'JavaScript'.

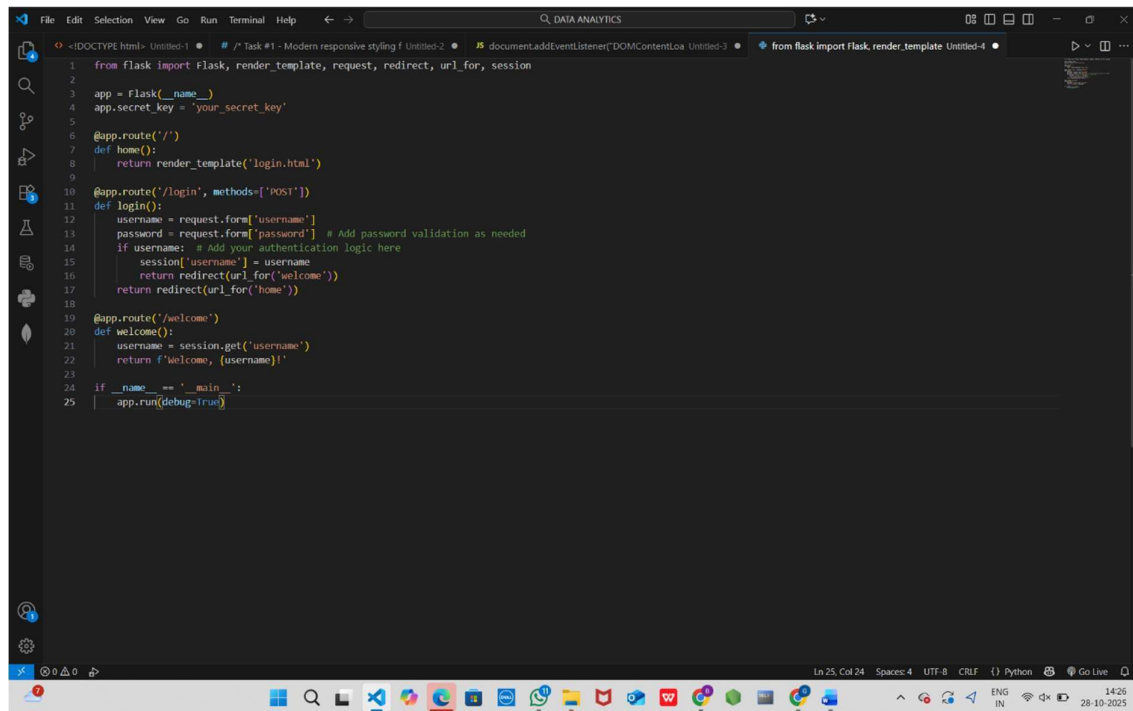
OUTPUT:

- *Working on submit JS validation.
- *Clear error messages if inputs are empty.

TASK 4: Ask AI to generate a Flask app that serves the HTML form (Task #3) and prints the username on successful login.

PROMPT: “Generate a Flask app that serves the login HTML form from Task #3 and displays the username on a successful login.”

CODE:

A screenshot of a Visual Studio Code editor window. The editor is open to a file named 'Untitled-4' which contains Python code for a Flask web application. The code includes imports for Flask, render_template, request, redirect, and session. It defines a Flask app, sets a secret key, and creates three routes: a home route, a login route with POST method, and a welcome route. The login route uses request.form to get username and password, and session to store the username. The welcome route uses session.get to retrieve the username. The code is line-numbered from 1 to 25. The bottom status bar shows 'Ln 25, Col 24', 'Spaces: 4', 'UTF-8', 'CRLF', 'Python', and 'Go Live'. The Windows taskbar is visible at the bottom with various application icons and a system clock showing 14:26 on 28-10-2025.

```
1 from flask import Flask, render_template, request, redirect, url_for, session
2
3 app = Flask(__name__)
4 app.secret_key = 'your_secret_key'
5
6 @app.route('/')
7 def home():
8     return render_template('login.html')
9
10 @app.route('/login', methods=['POST'])
11 def login():
12     username = request.form['username']
13     password = request.form['password'] # Add password validation as needed
14     if username: # Add your authentication logic here
15         session['username'] = username
16         return redirect(url_for('welcome'))
17     return redirect(url_for('home'))
18
19 @app.route('/welcome')
20 def welcome():
21     username = session.get('username')
22     return f'Welcome, {username}!'
23
24 if __name__ == '__main__':
25     app.run(debug=True)
```

OUTPUT:

- *Working on submit JS valida on.
- *Clear error messages if inputs are empty